

The velocity-magnitude derivative floor

Regularizing $\partial|\mathbf{u}|/\partial\mathbf{u}$ in the Exact-Newton Jacobian

Porous Navier–Stokes solver — implementation note

Code anchor. Julia parameter `PhysicalProperties.velocity_magnitude_derivative_floor` (default $\varepsilon_d = 10^{-12}$ in `config/base_config.json`); carried on the velocity-regularization policy `SmoothVelocityFloor` and consumed at three leaf sites: `dsigma_du` (`src/models/reaction.jl`), `compute_dtau_1_du` and `compute_dtau_2_du` (`src/stabilization/tau.jl`).

1 Where it arises

Both the reaction coefficient and the stabilization parameters depend on the velocity *only* through the Euclidean speed

$$|\mathbf{u}| = \sqrt{\mathbf{u} \cdot \mathbf{u}}, \quad (1)$$

namely

$$\sigma(\alpha, \mathbf{u}) = a(\alpha) + b(\alpha) |\mathbf{u}|, \quad \tau_{1,\text{NS}}^{-1} = \frac{c_1 \nu}{h^2} + \frac{c_2 |\mathbf{u}|}{h}, \quad (2)$$

with $\tau_1 = (\alpha \tau_{1,\text{NS}}^{-1} + \sigma)^{-1}$ and $\tau_2 = h^2 / (c_1 \alpha \tau_{1,\text{NS}})$.

The Exact-Newton tangent requires the directional derivatives $\partial\sigma/\partial\mathbf{u}$, $\partial\tau_1/\partial\mathbf{u}$ and $\partial\tau_2/\partial\mathbf{u}$, every one of which factors through the gradient of the speed,

$$\frac{\partial|\mathbf{u}|}{\partial\mathbf{u}} \cdot d\mathbf{u} = \frac{\mathbf{u} \cdot d\mathbf{u}}{|\mathbf{u}|}. \quad (3)$$

2 The singularity and its regularization

The map $\mathbf{u} \mapsto |\mathbf{u}|$ is only C^0 at the origin: (3) is the indeterminate $0/0$ at $\mathbf{u} = \mathbf{0}$. A stagnation point with $\mathbf{u} = \mathbf{0}$ is not exotic — it occurs exactly on perfect Dirichlet walls (and at interior stagnation points of a manufactured field), where the discrete velocity is pinned to zero. Evaluated verbatim, (3) returns NaN and poisons the whole Jacobian.

We regularize the denominator with a single additive floor $\varepsilon_d > 0$:

$$\frac{\partial|\mathbf{u}|}{\partial\mathbf{u}} \cdot d\mathbf{u} \approx \frac{\mathbf{u} \cdot d\mathbf{u}}{|\mathbf{u}| + \varepsilon_d}. \quad (4)$$

This is the only role of `velocity_magnitude_derivative_floor`. Concretely it enters

$$\partial_{\mathbf{u}}\sigma \cdot d\mathbf{u} = b(\alpha) \frac{\mathbf{u} \cdot d\mathbf{u}}{|\mathbf{u}| + \varepsilon_d} \quad (\text{dsigma_du, Forchheimer}), \quad (5)$$

$$\partial_{\mathbf{u}}\tau_{1,\text{NS}}^{-1} \cdot d\mathbf{u} = \frac{c_2}{h} \frac{\mathbf{u} \cdot d\mathbf{u}}{|\mathbf{u}| + \varepsilon_d} \quad (\text{inside compute_dtau_1_du, compute_dtau_2_du}), \quad (6)$$

with $\partial_{\mathbf{u}}\tau_1 = -\tau_1^2 (\alpha \partial_{\mathbf{u}}\tau_{1,\text{NS}}^{-1} + \partial_{\mathbf{u}}\sigma)$ and the analogous chain for τ_2 .

3 Why it cannot move the converged solution

Three facts pin down the status of ε_d as a *Jacobian-only* regularization:

1. **It is absent from the residual.** The nonlinear residual $\mathbf{R}(\mathbf{u}, p)$ that the solver drives to zero contains $\sigma \mathbf{u}$, τ_1 , τ_2 — but never (4). The fixed point $\mathbf{R} = \mathbf{0}$ is therefore independent of ε_d .
2. **It is dropped in Picard.** In `PicardMode` the coefficient derivatives $\partial_{\mathbf{u}}\sigma$, $\partial_{\mathbf{u}}\tau$ are frozen to structural zeros, so ε_d does not appear at all; the Picard tangent is exact in ε_d trivially.
3. **It is an inexact-Newton perturbation.** In `ExactNewtonMode` it perturbs the tangent by $O(\varepsilon_d/|\mathbf{u}|)$, which vanishes in the bulk where $|\mathbf{u}| \gg \varepsilon_d$ and is bounded near stagnation. A perturbed tangent changes only the Newton *convergence rate*, never the root it converges to.

Hence ε_d is a pure conditioning knob: it guarantees a finite, well-defined Jacobian everywhere while leaving the discrete solution — and thus every reported error and convergence rate — untouched.

4 Interaction with the speed floors (no double counting)

The speed handed to (4) is already the *regularized* speed produced by the velocity policy: `reaction_speed` $= \sqrt{\mathbf{u} \cdot \mathbf{u} + u_{\text{base}}^2}$ for σ , and `effective_speed` (which additionally carries a mesh-diffusive floor) for τ . Thus whenever the base floor $u_{\text{base}} = \text{u_base_floor_ref} > 0$ one already has $|\mathbf{u}| \geq u_{\text{base}} \gg \varepsilon_d$ and (4) is a no-op. This is in fact the situation in the convergence studies: the MMS harnesses set the *mesh-diffusive* weight `h_floor_weight` to 0 (so that the drag law stays mesh-independent) while inheriting $u_{\text{base}} = \text{u_base_floor_ref} = 10^{-4}$ from `config/base_config.json`; the configuration $u_{\text{base}} = 0$ with `h_floor_weight` = 0 is in fact rejected by the config validator, which requires the velocity floor to be strictly positive so that the regularization stays C^1 . Since $u_{\text{base}} = 10^{-4} \gg \varepsilon_d = 10^{-12}$ there, ε_d is by the criterion just stated a *no-op* in the convergence studies, and the errors they report are unaffected by it. It remains a genuine guard for configurations that switch the base floor off (e.g. `NoRegularization`), which is why it is retained. Keeping ε_d a *separate* parameter from `u\_base\_floor\_ref` and `epsilon_floor` (which guards an unrelated $h \rightarrow 0$ denominator) lets the three be reasoned about and tuned independently.

5 Recommended value

$\varepsilon_d = 10^{-12}$ (double precision near the rounding floor) is small enough to be numerically invisible wherever $|\mathbf{u}|$ is not itself at the level of machine epsilon, yet large enough to keep $\mathbf{u}/(|\mathbf{u}| + \varepsilon_d)$ finite. Because the parameter only conditions the Jacobian, any value in a wide band $\varepsilon_d \in [10^{-14}, 10^{-8}]$ gives indistinguishable solves; it is exposed as a config input purely so the value is documented, validated at load, and tunable rather than buried as a literal.